UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Proiect Prelucrare Grafica

OpenGl

Student:Berciu Oana-Georgiana

Grupa:30334

Laborant:Gorgan Raul

1.Cuprins

1. Cuprins
2. Prezentarea temei
3. Scenariul
   3.1. descrierea scenei și a obiectelor
   3.2. funcționalități
4. Detalii de implementare
   4.1. funcții și algortmi
      4.1.1. soluții posibile
      4.1.2. motivarea abordării alese
   4.2. modelul grafic
   4.3. structuri de date
   4.4. ierarhia de clase
5. Prezentarea interfeței grafice utilizator / manual de utilizare
6. Concluzii și dezvoltări ulterioare
7. Referințe

2.Prezentarea temei

Proiectul realizat în cadrul disciplinei Prelucrare Grafica constă în dezvoltarea unei scene 3D interactive utilizând OpenGL modern .Scopul proiectului este implementarea și integrarea principalelor concepte studiate pe parcursul laboratorului, precum iluminarea, texturarea, shadow mapping-ul și interacțiunea cu utilizatorul.

3.Scenariul

3.1Descrierea scenei

Scena reprezintă un mediu rural complex, inspirat din viața de la țară de odinioară, amplasat într-o zonă izolată, între dealuri și relief natural, departe de elementele moderne. Aceasta nu este o simplă fermă , ci un spațiu conceput pentru a sugera experiența vieții tradiționale, în care oamenii trăiau simplu, în strânsă legătură cu natura.

Utilizatorul poate explora liber scena și poate observa elemente specifice traiului rural: clădiri din lemn și cărămidă, animale domestice, zone de cultivare, felinare și obiecte utilitare. Mediul sugerează posibilitatea de campare, de îngrijire a animalelor și de culegere a plantelor, accentuând ideea unui stil de viață autosuficient, caracteristic vremurilor vechi, în care resursele erau limitate.

Scena este randată în timp real și include un ciclu zi/noapte, care schimbă atmosfera vizuală și modul de iluminare, contribuind la realism și la imersiunea utilizatorului în acest univers rural. Relieful, vegetația și iluminarea sunt gândite astfel încât să creeze o senzație de liniște, izolare și apropiere de natură.

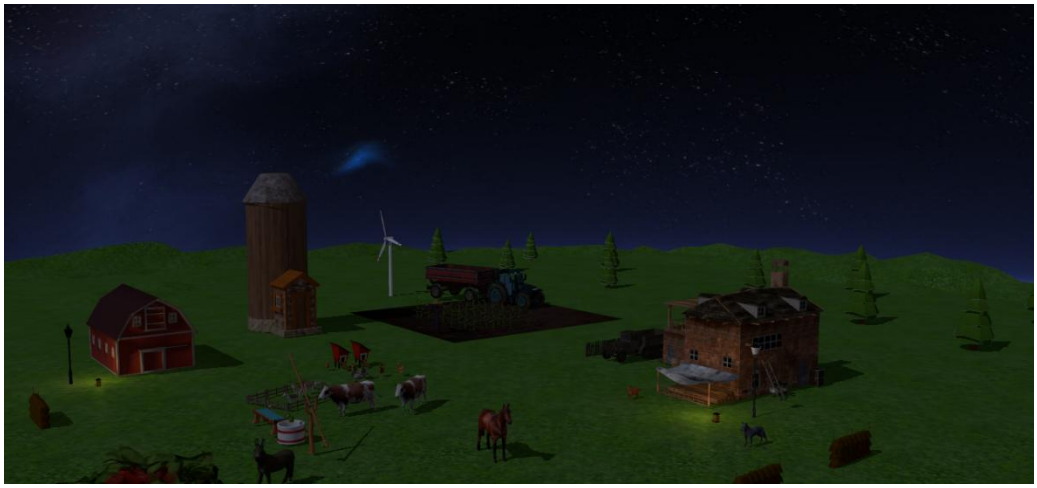
Modelele 3D au fost realizate sau preluate din surse externe și integrate în scenă folosind formatul .obj, împreună cu fișierele de materiale (.mtl) și texturi.

3.2 Functionalitati

Aplicația oferă următoarele funcționalități:

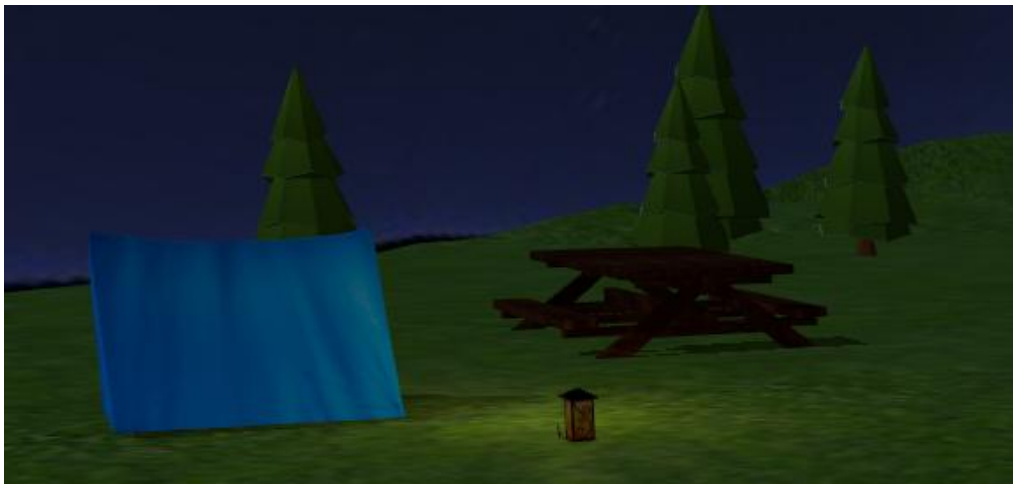
- **Navigare liberă în scenă**
  Utilizatorul poate explora mediul 3D folosind tastatura pentru deplasare și mouse-ul pentru orientarea camerei.

- **Mod zi / noapte**
  Scena permite comutarea între modul zi și modul noapte, modificând culoarea iluminarii si schimband obiectul ski.



- **Iluminare direcțională (lumina globală – tip „soare”)**
  O sursă de lumină direcțională simulează lumina solară și asigură iluminarea generală a scenei în modul zi.

- **Iluminare punctiformă (felinare)**
  Felinarele sunt simulate prin surse de lumină punctiforme, care iluminează radial zona din jurul lor, fiind active în special în modul noapte.



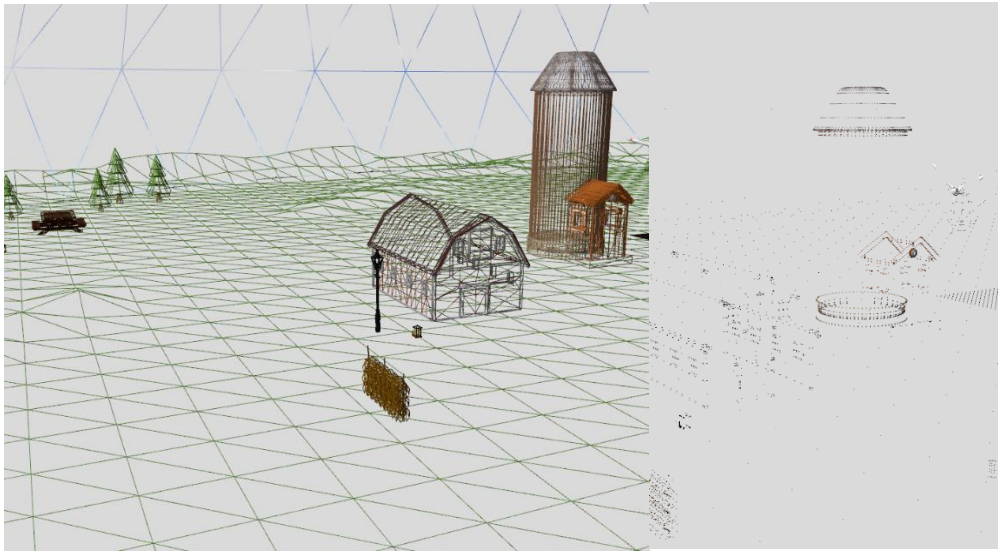
- **Ceață (Fog)**

Utilizatorul poate activa sau dezactiva efectul de ceață din tastatură, pentru a spori realismul și senzația de profunzime a scenei.De asemenea in modul de noaptea culoarea cetei se schimba pentru un efect de realism.

- **Shadow mapping**
  Aplicația utilizează tehnica shadow mapping pentru redarea umbrelor dinamice, în funcție de poziția surselor de lumină și a obiectelor din scenă.

- **Animație de prezentare automată a scenei**
  La pornirea aplicației, camera parcurge automat scena pe o traiectorie prestabilită, oferind o prezentare generală a mediului.

- **Moduri de vizualizare a scenei**
  Scena poate fi afișată în diferite moduri de randare:solid, wireframe, poligonal (point) și smooth, permițând analiza structurii geometrice a obiectelor.



- **Animație pentru obiect**
  Pe tot parcursul rularii aplicației, obiectul windmill se invarte,simuland vantul.

4.Detalii de implementare

Acest capitol descrie soluțiile tehnice utilizate pentru realizarea aplicației, structura internă a programului, algoritmii implementați și modul în care sunt organizate datele și clasele. Implementarea este realizată în C++ folosind OpenGL modern, împreună cu biblioteci auxiliare precum GLFW, GLEW și GLM.

4.1 Funcții și algoritmi

4.1.1 Soluții posibile

Pentru implementarea aplicației au fost luate în considerare următoarele soluții:

- Gestionarea camerei
  - cameră controlată exclusiv de utilizator;
  - combinație între cameră animată automat și control manual.
- Randarea scenei
  - randare simplă, fără efecte avansate;
  - randare cu iluminare de bază (o singură sursă de lumină);
  - randare avansată cu surse multiple de lumină și umbre dinamice.
- Iluminare
  - iluminare direcțională pentru simularea soarelui;
  - iluminare mixtă: direcțională + surse punctiforme.
- Umbre
  - umbre statice;
  - umbre dinamice utilizând shadow mapping.
- Prezentarea scenei
  - prezentare automată a scenei printr-o animație a camerei.

4.1.2 Motivarea abordării alese

Pentru acest proiect a fost aleasă o soluție care combină interactivitatea cu realismul vizual.

Camera utilizează o animație automată de prezentare, realizată prin interpolarea poziției și direcției între puncte prestabilite, urmată de preluarea controlului de către utilizator. Această abordare permite o introducere vizuală a scenei, urmată de explorare liberă.

Iluminarea este realizată printr-o sursă de lumină direcțională, care simulează lumina solară, și mai multe surse de lumină punctiforme, utilizate pentru iluminarea locală a felinarelor. Această soluție oferă un echilibru între performanță și realism.

Pentru redarea umbrelor a fost aleasă tehnica de shadow mapping, care permite generarea umbrelor dinamice în timp real, în funcție de poziția luminii și a obiectelor din scenă.

Pentru sporirea realismului vizual a fost implementat un efect de ceață (fog), care poate fi activat sau dezactivat de utilizator, precum și un mod zi/noapte, care modifică parametrii de iluminare și atmosfera scenei.

4.2 Modelul grafic

Modelul grafic al aplicației descrie modul în care scena 3D este construită din punct de vedere vizual, precum și modul în care obiectele sunt reprezentate, poziționate și randate în OpenGL.

Scena este alcătuită din mai multe modele 3D statice și animate, încărcate din fișiere externe în format .obj, împreună cu fișierele de materiale (.mtl) și texturile asociate. Modelele sunt realizate sau preluate din surse externe și integrate în aplicație folosind o clasă dedicată pentru încărcare și randare.

Fiecare obiect din scenă este definit printr-o matrice de model, care include transformări de:

- translație, pentru poziționarea obiectelor în scenă;
- rotație, utilizată pentru orientarea obiectelor și pentru animații;
- scalare, pentru adaptarea dimensiunilor modelelor la scara scenei.

Camera utilizează un sistem de coordonate tridimensional, iar scena este vizualizată prin intermediul unei matrici de vizualizare (view) și al unei matrici de proiecție perspectivă, care oferă senzația de adâncime și realism.

Iluminarea este calculată în shader, folosind vectori de lumină transformați din spațiul world în spațiul eye, iar pentru fiecare obiect este calculată o matrice normală, necesară pentru iluminarea corectă a suprafețelor.

Pentru redarea umbrelor dinamice, scena este randată suplimentar din perspectiva sursei de lumină, obținându-se o hartă de adâncime (depth map) utilizată ulterior în procesul de randare final.

Animațiile din scenă sunt realizate prin actualizarea continuă a matricilor de transformare, cum este cazul palelor morii de vânt, care se rotesc în jurul axei proprii, simulând funcționarea reală a mecanismului.

4.3 Structuri de date

Pentru organizarea și gestionarea eficientă a informațiilor din aplicație au fost utilizate mai multe structuri de date, adaptate cerințelor unei scene 3D interactive.

1. Structura CameraPoint este utilizată pentru definirea traiectoriei camerei în timpul animației de prezentare automată a scenei. Aceasta conține:

- poziția camerei în spațiul 3D;
- punctul spre care este orientată camera;
- durata de timp aferentă fiecărui segment de animație.

Prin utilizarea unui vector de astfel de structuri, aplicația poate interpola poziția și direcția camerei între mai multe puncte, obținând o mișcare fluentă și continuă.

2. Structura PointLight descrie o sursă de lumină punctiformă din scenă. Aceasta include:

- poziția sursei de lumină;
- culoarea luminii;
- coeficienții de atenuare (constant, liniar și quadratic).

Aceste date sunt transmise către shader pentru calculul iluminării locale, fiind utilizate în special pentru simularea felinarelor active în modul noapte.

3 .Pentru reprezentarea transformărilor geometrice și a calculelor matematice au fost utilizate structurile oferite de biblioteca GLM:

- glm::vec3 – pentru poziții, direcții și culori;
- glm::mat4 – pentru matricile de transformare (model, view, projection);
- glm::mat3 – pentru matricea normală utilizată în calculul iluminării.

Aceste structuri permit efectuarea eficientă a operațiilor matematice necesare randării 3D și asigură compatibilitatea cu pipeline-ul OpenGL modern.

4.4 Ierarhia de clase

Aplicația este organizată pe clase, conform principiilor programării orientate pe obiect, fiecare clasă având o responsabilitate clară în cadrul procesului de randare și interacțiune. Această organizare permite o separare logică a funcționalităților și facilitează întreținerea și extinderea proiectului.

Clasa Window gestionează crearea și distrugerea ferestrei OpenGL și inițializarea contextului grafic. Funcția Create() inițializează GLFW, configurează versiunea OpenGL utilizată (4.1 Core Profile), activează suportul pentru sRGB, antialiasing și afișare HiDPI, apoi creează fereastra aplicației și setează contextul curent.

Clasa Shader este responsabilă de încărcarea, compilarea și utilizarea shaderelor OpenGL. Aceasta citește codul sursă al shaderelor din fișiere externe, compilează shaderul de tip vertex și fragment și le leagă într-un program shader utilizat la randare.

Clasa Model3D este responsabilă de încărcarea, gestionarea și randarea modelelor 3D din scenă. Aceasta abstractizează procesul de citire a fișierelor .obj, împreună cu materialele și texturile asociate, permițând utilizarea facilă a modelelor în aplicație.

Clasa Camera gestionează poziția și orientarea punctului de observație în scena 3D. Aceasta permite deplasarea și rotirea camerei, oferind utilizatorului posibilitatea de a explora scena din perspective diferite.

Camera este definită prin poziție, direcția de privire, direcția „up” și direcția laterală, toate calculate pe baza vectorilor tridimensionali. Matricea de vizualizare este obținută folosind funcția lookAt, fiind utilizată ulterior în procesul de randare.

Clasa oferă metode pentru deplasarea camerei în scenă (înainte, înapoi, stânga, dreapta) și pentru rotirea acesteia în funcție de mișcarea mouse-ului, prin intermediul unghiurilor yaw și pitch. De asemenea, permite accesul la poziția curentă a camerei.

Această clasă este esențială pentru interacțiunea utilizatorului cu scena și pentru controlul perspectivei din care este vizualizat mediul 3D.

Clasa main este fișierul principal al aplicației coordonează inițializarea, execuția și închiderea aplicației grafice. Acesta utilizează clasele definite anterior (Window, Shader, Camera, Model3D) și le integrează într-un flux unitar de randare și interacțiune.

În cadrul acestui modul sunt realizate:

- inițializarea ferestrei OpenGL și a stării inițiale a contextului grafic;
- încărcarea modelelor 3D și a shaderelor;
- configurarea iluminării, umbrelor și a efectelor vizuale;
- gestionarea inputului de la tastatură și mouse;
- randarea continuă a scenei în bucla principală a aplicației.

Bucla principală a aplicației asigură actualizarea animațiilor, procesarea inputului utilizatorului și afișarea scenei în timp real, până la închiderea ferestrei. La final, resursele alocate sunt eliberate corespunzător.

Acest modul nu definește o clasă distinctă, ci reprezintă punctul central de control al aplicației, coordonând interacțiunea dintre toate componentele sistemului.

Clasa Mesh reprezintă componenta de bază pentru randarea geometrică a obiectelor 3D. Aceasta gestionează datele geometrice ale unui obiect, respectiv vertecșii, indicii și texturile asociate, precum și buffer-ele OpenGL necesare desenării acestuia.

Clasa se ocupă de inițializarea buffer-elor de tip VAO, VBO și EBO și de definirea atributelor de vertecși (poziție, normale și coordonate de textură). De asemenea, Mesh aplică texturile corespunzătoare și realizează apelul de desenare folosind primitive de tip triunghiuri.

Această clasă este utilizată intern de clasa Model3D, permițând randarea eficientă a modelelor complexe formate din mai multe subcomponente.

5.Manual utlizare

Interacțiunea cu scena se realizeaza exclusiv prin tastatură și mouse.

Control camera:

- **W** – deplasare înainte

- **S** – deplasare înapoi
- **A** – deplasare spre stânga
- **D** – deplasare spre dreapta
- **Mouse** – rotirea camerei (schimbarea direcției de privire)

Control iluminare:

- **L** – rotirea luminii direcționale (simularea mișcării soarelui pe cer)
- **J** – rotirea luminii direcționale în sens opus
- **N** – activare / dezactivare mod noapte
  (iluminarea globală isi schimba culoarea, iar felinarele devin sursa principală de lumină)

Efecte vizuale

- **C** – activare / dezactivare ceață (fog)

Moduri de randare

- **1** – randare solid
- **2** – randare wireframe
- **3** – randare poligonală (point)

6. Concluzii și dezvoltări ulterioare

Proiectul demonstrează utilizarea conceptelor fundamentale de grafică 3D în OpenGL. Scena este interactivă, bine iluminată și include efecte vizuale,ceea ce o face extrem de realista.

Dezvoltări ulterioare posibile:

- introducerea de animații pentru animale, pentru creșterea realismului;
- adăugarea de sunete ambientale specifice mediului rural;

7.Referinte

- LearnOpenGL – *Lighting & Shadow Mapping* - https://learnopengl.com/
- Laboratoarele de Prelucrare Grafică
- Blender – *Open-source 3D creation suite* - https://docs.blender.org/
- Sketchfab – platformă de modele 3D- https://sketchfab.com/feed
- Free3D - platformă de modele 3D - https://free3d.com/
- TurboSquid – platformă de modele 3D - https://www.turbosquid.com/